LABORATORY RECORD

Course: Full Stack Web Development
Course Code: 23CM4121
Experiment No.: 6
Student Name: N. Samhavya
Roll No.: A24126552099
Date: 29-08-2026

---

1. TITLE

Implement Modules in Node.js

---

2. AIM

To understand and implement **built-in, user-defined, and third-party modules in Node.js** and demonstrate their usage through file handling, operating system information, path manipulation, HTTP server creation, URL parsing, events, and Express.js.

---

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept of modules in Node.js.
- To use built-in Node.js modules.
- To perform file operations using the `fs` module.
- To obtain operating system information using the `os` module.
- To manipulate file paths using the `path` module.
- To create a basic HTTP server using the `http` module.
- To parse URLs using the `url` module.
- To create and handle custom events using the `events` module.
- To understand user-defined modules using `module.exports`. To import and use functions from a user-defined module.
- To understand the use of the Express.js module for creating a web server.

4. THEORY

Node.js provides a modular architecture that allows applications to be divided into smaller, reusable components called **modules**. Modules help organize code and make applications easier to maintain and reuse.
Node.js modules can be broadly classified into:

1. **Built-in Modules**
2. **User-defined Modules**
3. **Third-party Modules**

Built-in Modules

Node.js provides several built-in modules that can be directly imported using the require() function.
The following built-in modules are demonstrated in this experiment:

- **fs** – Used for file system operations such as creating and writing files.
- **os** – Provides information about the operating system and CPU.
- **path** – Provides utilities for working with file and directory paths.
- **http** – Used to create HTTP servers.
- **url** – Used to parse and work with URLs.
- **events** – Provides the EventEmitter class for creating and handling custom events.

User-defined Module

A user-defined module is a JavaScript file created by the developer. Functions or variables can be exported using module.exports and imported into another file using require(). In this experiment, arithmetic functions such as **addition, subtraction, multiplication, and division** are exported from a user-defined module.

Express Module

Express.js is a third-party web framework for Node.js. It simplifies the process of creating web servers and handling HTTP requests and responses.
In the Express program, a GET route is created for /, which sends the message **"Hello from Express!"** to the browser.
The basic module working flow is:
Module Creation / Built-in Module → require() → Functionality → Output

5. ALGORITHM / PROCEDURE

1. Create a new Node.js project.
2. Initialize the project using npm init.
3. Import the required built-in modules using require().
4. Use the fs module to create and write data into a text file.
5. Use the os module to display the operating system platform and CPU core count.
6. Use the path module to obtain the base name of a file path.
7. Use the http module to create a basic Node.js HTTP server.
8. Start the HTTP server on port 3000.
9. Use the url module to create and parse a URL.
10. Display the protocol, host, path, and query parameters.

11. Import the EventEmitter class from the events module.
12. Create a custom event using EventEmitter.
13. Register an event listener and trigger the event.
14. Create a user-defined module containing arithmetic functions.
15. Export the functions using module.exports.
16. Import the user-defined module using require().
17. Use the exported functions in another JavaScript file.
18. Install Express.js using npm install express.
19. Create an Express application.
20. Define a GET route and start the Express server.
21. Execute the programs and verify the output.

---

6. PROGRAM

Program 1: Built-in Modules

builtIn.js

```
//fs module const fs = require("fs");
fs.writeFileSync("hello.txt","hello node.js");
console.log("File created");
//os module const os = require("os");
console.log(os.platform()); //os which we are
using console.log(os.cpus().length); //no. of cpu
cores
//path module const path = require("path");
console.log(path.basename("C:/Users/anits/Desktop/A24126552126/week5/hello.txt"));
// http module const http = require("http");
const server = http.createServer((req, res) =>
{    res.end("Hello from Node.js!");
});
server.listen(3000); console.log("Server
running on port 3000");
// url module const { URL } = require("url"); const myURL = new
URL("https://example.com:8080/products?id=101");
console.log("Protocol:", myURL.protocol); console.log("Host:",
myURL.host); console.log("Path:", myURL.pathname);
console.log("Query:", myURL.search);
//events module const EventEmitter =
require("events"); const myEvent =
```

```
new EventEmitter();
myEvent.on("welcome", () =>
{    console.log("Welcome to Node.js!");
});
myEvent.emit("welcome");
```

Program 2: Express Module

module.js

```
const express = require("express");
const app = express(); app.get("/",
(req, res) => {    res.send("Hello
from Express!");
});
app.listen(3000, () => {    console.log("Server
running on port 3000");
});
```

Program 3: User-defined Module

app.js

```
// exporting user defined module
function add(a, b) {    return a +
b;
}
function
sub(a,b){    return a-b;
}
function
mul(a,b){    return a*b;
}
function
div(a,b){    return a/b;
}
module.exports =
{    add, sub, mul, div
};
```

user.js

```
const math = require("./app");
```

```
console.log("Add:", math.add(10,5));
console.log("Sub:",  math.sub(10,5));
console.log("Mul",  math.mul(10,5));
console.log("Div", math.div(10,5));
```

Supporting File: hello.txt

The fs module creates the file hello.txt and writes the following content into it:

```
hello node.js
```

---

7. CODE EXPLANATION

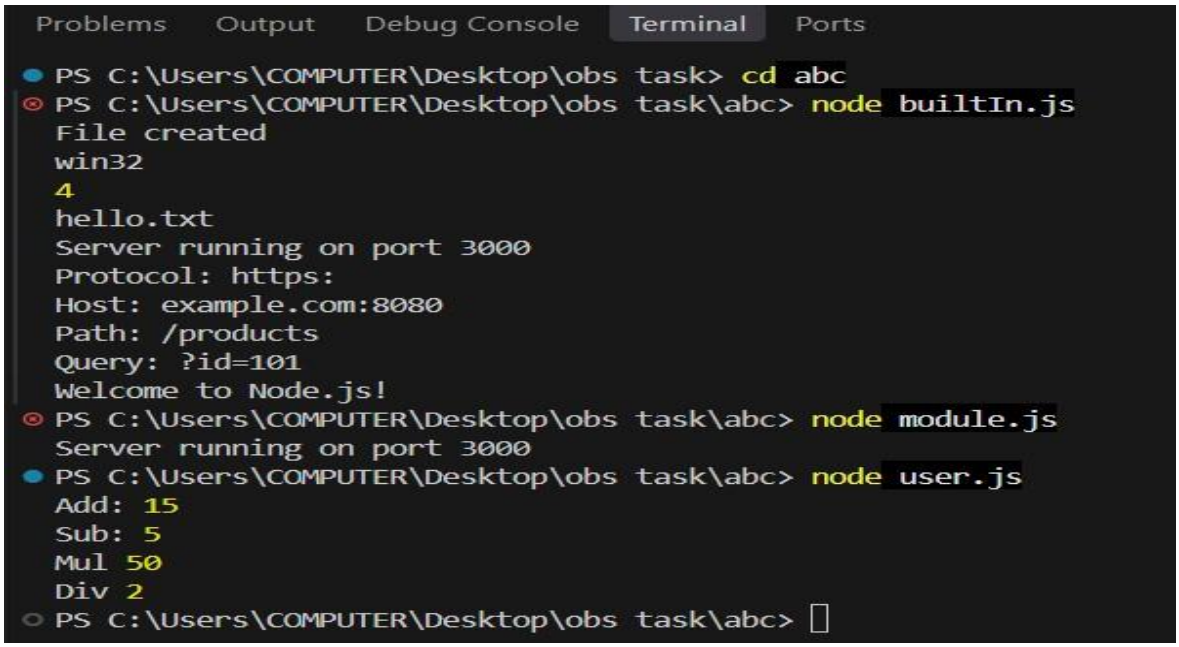
Code / Concept	Explanation
require("fs")	Imports the built-in File System module.
writeFileSync()	Creates or overwrites a file and writes data synchronously.
hello.txt	Text file created by the fs module.
require("os")	Imports the Operating System module.
os.platform()	Returns information about the operating system platform.
os.cpus()	Returns information about the available CPU cores.
require("path")	Imports the Path module.
path.basename()	Returns the final portion of a file path.
require("http")	Imports the HTTP module.
http.createServer()	Creates an HTTP server.
res.end()	Sends a response and ends the HTTP response.
server.listen(3000)	Starts the HTTP server on port 3000.
require("url")	Imports URL-related functionality.
new URL()	Creates a URL object for parsing URL components.
protocol	Returns the protocol portion of the URL.
host	Returns the hostname and port.
pathname	Returns the path portion of the URL.
search	Returns the query string of the URL.
require("events")	Imports the Node.js Events module.
EventEmitter	Used to create and manage custom events.
.on()	Registers an event listener.
.emit()	Triggers an event.
require("express")	Imports the Express.js framework.
express()	Creates an Express application.
app.get()	Defines a handler for HTTP GET requests.
res.send()	Sends a response to the client.
app.listen()	Starts the Express server.
module.exports	Exports functions or objects from a user-defined module.
add()	Returns the sum of two numbers.
sub()	Returns the difference of two numbers.
mul()	Returns the product of two numbers.
div()	Returns the quotient of two numbers.

8. TEST CASES AND EXECUTION DATA

The Node.js programs were executed and the functionality of the built-in, user-defined, and Express modules was verified.

Test Case	TC-01
Test / Input	Execute `builtin.js` using Node.js
Expected Result	The program should successfully demonstrate the built-in Node.js modules such as `fs`, `os`, `path`, `http`, and `url`.
Actual Result	The program successfully created a file, displayed operating system information, file information, started the server on port 3000, and displayed URL details.
Status	**Pass**

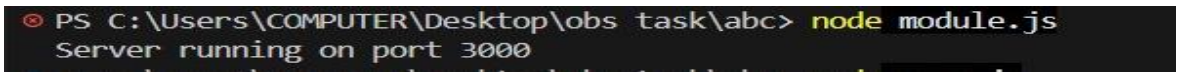
TC-01 Output:



Test Case	TC-02
Test / Input	Execute `module.js` using Node.js
Expected Result	The Express.js module should successfully create and run a web server on port 3000.
Actual Result	The server was started successfully and displayed `Server running on port 3000`.

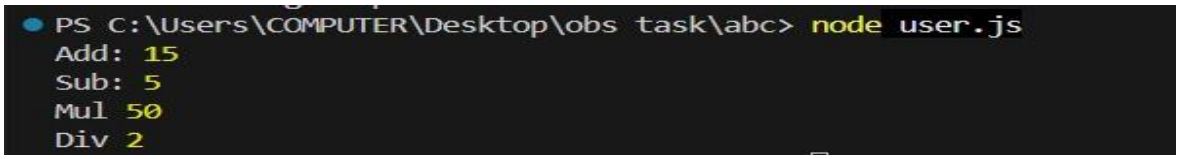
Status	Pass

TC-02 Output:



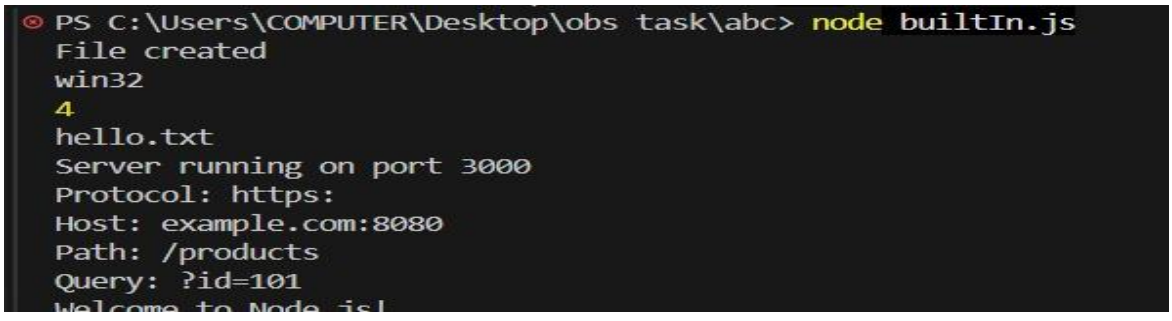
Test Case	TC-03
Test / Input	Execute `user.js` using Node.js
Expected Result	The user-defined module should successfully perform the arithmetic operations and display the results for addition, subtraction, multiplication, and division.
Actual Result	The program successfully displayed `Add: 15`, `Sub: 5`, `Mul: 50`, and `Div: 2`.
Status	**Pass**

TC-03 Output:



Test Case	TC-04
Test / Input	Verify file system operations performed by `builtin.js`
Expected Result	The program should create a file and display the file name and related file system output correctly.
Actual Result	The file was created successfully and `hello.txt` was displayed in the output.
Status	**Pass**

TC-04 Output:



Test Case	TC-05
Test / Input	Verify URL module output in `builtin.js`
Expected Result	The program should correctly display the protocol, host, path, and query values from the URL.
Actual Result	The output correctly displayed `https:`, `example.com:8080`, `/products`, and `pid=101`.

Status	Pass

TC-05 Output:

```
Server running on port 3000
Protocol: https:
Host: example.com:8080
Path: /products
Query: ?id=101
Welcome to Node.js!
```

---

9. OUTPUT

Output: Node.js Modules Execution

The following screenshot shows the execution output of the **Node.js modules programs**. It includes the outputs generated by the built-in modules such as fs, os, path, http, url, and events, along with the corresponding Node.js/Express execution results.

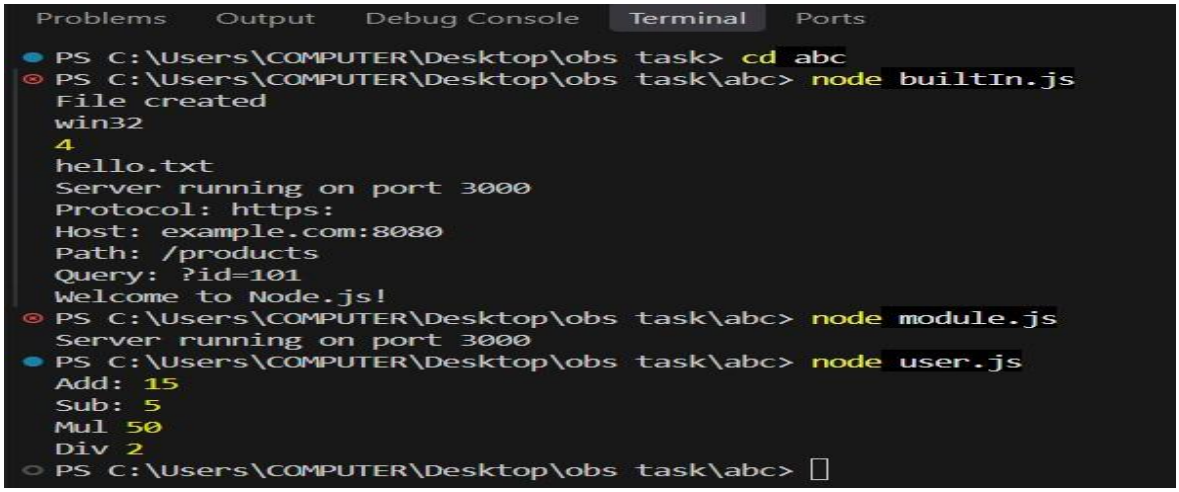


Figure 1: Output of Modules Implementation Using Node.js

10. RESULT

Thus, the different types of **Node.js modules** were successfully implemented and executed. Built-in modules such as fs, os, path, http, url, and events were used successfully. A userdefined module was created using module.exports, and the Express.js module was used to create a basic web server. The programs were executed successfully and the expected results were obtained.